

# Matrices and Circuit Equations

*Engineering Computation with Python* – Chapter 9

---

Muním Zabidi

Faculty of Electrical Engineering, Universiti Teknologi Malaysia

**1** Why Circuits Need Matrices

**2** Matrix Operations in Detail

**3** Matrix vs. Array Arithmetic

**4** Worked Matrix Examples

**5** Complex Number Arithmetic

**6** Summary and Practice

## 1 **Why Circuits Need Matrices**

---

## 2 **Matrix Operations in Detail**

---

## 3 **Matrix vs. Array Arithmetic**

---

## 4 **Worked Matrix Examples**

---

## 5 **Complex Number Arithmetic**

---

## 6 **Summary and Practice**

---

A circuit with resistors  $R_1, R_2, R_3$  and sources  $V_1, V_2$ . Kirchhoff's voltage law around each loop gives two equations in the two unknown loop currents  $I_1, I_2$ :

$$\begin{aligned}(R_1 + R_2) I_1 - R_2 I_2 &= V_1, \\ -R_2 I_1 + (R_2 + R_3) I_2 &= V_2.\end{aligned}$$

## The problem

Solvable by substitution for two loops. Tedious for three. **Impractical** for ten.

All the information sits in a grid of coefficients and a column of source voltages:

$$\underbrace{\begin{bmatrix} R_1 + R_2 & -R_2 \\ -R_2 & R_2 + R_3 \end{bmatrix}}_{\mathbf{A}} \underbrace{\begin{bmatrix} I_1 \\ I_2 \end{bmatrix}}_{\mathbf{x}} = \underbrace{\begin{bmatrix} V_1 \\ V_2 \end{bmatrix}}_{\mathbf{b}}.$$

$$\mathbf{Ax} = \mathbf{b}$$

**A** holds what the circuit **is**. **b** holds what **drives** it. **x** holds what you want to **find**.

This pattern — one matrix equation standing in for many simultaneous equations — is one of the most important in all of engineering.

No need to invert **A** by hand, no need to rearrange the equations:

```
R1, R2, R3 = 100.0, 220.0, 330.0    # ohms
V1, V2 = 12.0, 5.0                 # volts

A = np.array([[R1 + R2, -R2],
              [-R2,    R2 + R3]])
b = np.array([V1, V2])

I = np.linalg.solve(A, b)
print(f"I1 = {I[0]*1000:.3f} mA")
print(f"I2 = {I[1]*1000:.3f} mA")
```

I1 = 60.345 mA    I2 = 33.229 mA

## Check

Substitute back — **always**:

```
print(A @ I)
# Output: [12.  5.]
```

Matches  $V_1$ ,  $V_2$ . This is the single most valuable habit in numerical linear algebra.

## Interpret

Both currents positive: both loop currents flow in the assumed direction. A negative result would just mean the true current flows the other way.

Tens of mA is reasonable for this circuit.

## Predict before you run

If  $R_2$  — shared between the loops — were made very large, what happens to  $I_1$  and  $I_2$ ? Predict, then set  $R_2 = 1\text{e}6$  and check.

1 Why Circuits Need Matrices

2 Matrix Operations in Detail

3 Matrix vs. Array Arithmetic

4 Worked Matrix Examples

5 Complex Number Arithmetic

6 Summary and Practice

$$\begin{aligned} 3x_1 + 2x_2 &= 12, \\ x_1 + 4x_2 &= 10, \end{aligned} \quad \Longleftrightarrow \quad \begin{bmatrix} 3 & 2 \\ 1 & 4 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 12 \\ 10 \end{bmatrix}.$$

Solving means finding the single vector  $\mathbf{x}$  that satisfies **all** the equations at once:  $x_1 = 2.8, x_2 = 1.8$ .

## What changes, what doesn't

The matrix form does not change the arithmetic; it changes the **bookkeeping** — a procedure written once works for two unknowns or two hundred.

# Why Not Just Invert the Matrix?

Algebraically,  $\mathbf{x} = \mathbf{A}^{-1}\mathbf{b}$  is correct. It is nevertheless the **wrong** way to compute the answer:

## It is slower

Forming the full inverse computes far more than the problem requires. Solving directly does roughly a third of the work for a large system.

## It is less accurate

Inversion introduces rounding at every step, which then propagates through the multiplication by  $\mathbf{b}$ .

## The rule throughout this course

Use `np.linalg.solve(A, b)`, never `np.linalg.inv(A) @ b`.

# When There Is No Solution

If one equation is a multiple of another:

$$\begin{bmatrix} 1 & 2 \\ 2 & 4 \end{bmatrix}$$

the two rows carry the same information — no unique  $\mathbf{x}$  exists. This matrix is **singular**; its determinant is zero.

```
S = np.array([[1.0, 2.0],  
              [2.0, 4.0]])  
b = np.array([1.0, 2.0])  
  
print(np.linalg.det(S))      # 0.0 -- singular  
x = np.linalg.solve(S, b)    # raises LinAlgError
```

## In a circuit problem

A singular  $\mathbf{A}$  almost always means the equations were set up wrongly — not that the circuit is impossible. The error is a useful early warning.

$$m = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$$

Computed element by element:

| Result    | Calculation                   | Value |
|-----------|-------------------------------|-------|
| $R_{0,0}$ | $(1 \times 1) + (2 \times 3)$ | 7     |
| $R_{0,1}$ | $(1 \times 2) + (2 \times 4)$ | 10    |
| $R_{1,0}$ | $(3 \times 1) + (4 \times 3)$ | 15    |
| $R_{1,1}$ | $(3 \times 2) + (4 \times 4)$ | 22    |

$$m \times m = \begin{pmatrix} 7 & 10 \\ 15 & 22 \end{pmatrix}$$

# Transpose, Right Division, Left Division

## Transpose

Flips a matrix over its main diagonal: rows become columns.

$$a = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix} \rightarrow a^T = \begin{pmatrix} 1 & 3 \\ 2 & 4 \end{pmatrix}$$

## Right division

Find  $X$  solving  $XA = B$ :

$$X = BA^{-1}$$

## Left division

Find  $X$  solving  $AX = B$ :

$$X = A^{-1}B$$

## Order matters

Matrix multiplication is **not** commutative:  $BA^{-1} \neq A^{-1}B$ . Right division and left division solve different equations.

In practice, neither Python nor other languages compute  $A^{-1}$  explicitly — a numerically stable decomposition solves for  $X$  directly.

Raising a square matrix  $A$  to an integer power  $n$  means repeated **matrix** multiplication:

$$A^n = \underbrace{A \times A \times \cdots \times A}_{n \text{ times}}.$$

Not the same as element-wise power

$A^n$  (matrix power) and raising every **element** of  $A$  to the power  $n$  generally give different results, and are written differently in Python.

1 Why Circuits Need Matrices

2 Matrix Operations in Detail

3 **Matrix vs. Array Arithmetic**

4 Worked Matrix Examples

5 Complex Number Arithmetic

6 Summary and Practice

## Matrix operations

Use `@`, or functions such as `np.dot()`, `np.matmul()`, `np.linalg.solve()`.

## Element-wise (array) operations

Standard operators `+` `-` `*` `/` act on each corresponding element, and work on arrays of any shape.

## They agree on two operations

Addition (`+`) and subtraction (`-`) behave **identically** whether you mean “matrix” or “element-wise.” Only multiplication, division, and power differ.

As with ordinary arithmetic, NumPy follows an order of precedence:

| Order | Operation                                |
|-------|------------------------------------------|
| 1     | Parentheses                              |
| 2     | Power                                    |
| 3     | Transpose                                |
| 4     | Multiplication & Division, left to right |
| 5     | Addition & Subtraction                   |
| 6     | Colon : (slice)                          |

| Algebra                      | Matrix (Python)                | Array (Python)      |
|------------------------------|--------------------------------|---------------------|
| $a + b$                      | <code>a + b</code>             | <code>a + b</code>  |
| $a - b$                      | <code>a - b</code>             | <code>a - b</code>  |
| $a \times b$                 | <code>a @ b</code>             | <code>a * b</code>  |
| $a/b$ (right div.)           | <code>solve(a.T,b.T).T</code>  | <code>a / b</code>  |
| $a \backslash b$ (left div.) | <code>solve(a, b)</code>       | <code>b / a</code>  |
| $a^n$                        | <code>matrix_power(a,n)</code> | <code>a ** n</code> |
| $a^T$                        | <code>a.T</code>               | <code>a.T</code>    |

## A note on `np.matrix`

Legacy and not used in this course – always `np.array` with `@`. Appendix B records the differences.

1 Why Circuits Need Matrices

2 Matrix Operations in Detail

3 Matrix vs. Array Arithmetic

4 **Worked Matrix Examples**

5 Complex Number Arithmetic

6 Summary and Practice

```
A = np.array([[1, 2], [3, 4]])  
B = np.array([[5, 6], [7, 8]])  
  
A + B      # [[ 6  8] [10 12]]  
A - B      # [[-4 -4] [-4 -4]]  
A @ B      # [[19 22] [43 50]]  
np.matmul(A, B)  # same as A @ B
```

## Notice

Addition and subtraction look exactly like array arithmetic. Multiplication does not — @ is required for the true matrix product.

```
# Right division: solving  $XA = B$ 
np.linalg.solve(A.T, B.T).T
# [[-1.  2.] [-2.  3.]]

# Left division: solving  $AX = B$ 
np.linalg.solve(A, B)
# [[-3. -4.] [ 4.  5.]]

# Matrix power
np.linalg.matrix_power(A, 2)
# [[ 7 10] [15 22]]

# Transpose
A.T  # [[1 3] [2 4]]
```

# Precedence in a Complex Expression

The rules from the previous section govern how Python evaluates a compound matrix expression:

- **Transpose before multiplication:**  $A.T @ B$
- **Parentheses force order:**  $(A + B) @ B$
- **Power before multiplication:** `matrix_power(A,2) @ B`
- **Multiplication before addition:**  $A @ B + B$

## Worked check

$$A.T @ B = \begin{pmatrix} 26 & 30 \\ 38 & 44 \end{pmatrix} \quad (A+B) @ B = \begin{pmatrix} 86 & 100 \\ 134 & 156 \end{pmatrix}$$

```
# Power, transpose, addition, and division together
result = (np.linalg.matrix_power(A, 2) + A.T) \
          @ np.linalg.solve(B, A)
print(result)
```

```
[[ -12.   -7.]  [-19.  -10.]]
```

## Reading it left to right

Power first (`matrix_power(A, 2)`), then transpose (`A.T`), then their sum, then `solve(B, A)` (left division), then the final matrix product.

|   |                                         |
|---|-----------------------------------------|
| 1 | <u>Why Circuits Need Matrices</u>       |
| 2 | <u>Matrix Operations in Detail</u>      |
| 3 | <u>Matrix vs. Array Arithmetic</u>      |
| 4 | <u>Worked Matrix Examples</u>           |
| 5 | <u><b>Complex Number Arithmetic</b></u> |
| 6 | <u>Summary and Practice</u>             |

# Pure Transpose vs. Conjugate Transpose

With complex-valued arrays, two different “transposes” exist:

```
a = np.array([[1+1j, 2+2j], [3+3j, 4+4j]])

a.T          # pure transpose -- swaps rows/cols only
a.conj().T   # conjugate (Hermitian) transpose
```

a.T:

$$\begin{pmatrix} 1+1j & 3+3j \\ 2+2j & 4+4j \end{pmatrix}$$

a.conj().T:

$$\begin{pmatrix} 1-1j & 3-3j \\ 2-2j & 4-4j \end{pmatrix}$$

## The difference

.T swaps rows and columns, leaving every real and imaginary part unchanged. .conj().T **additionally** negates every imaginary part.

1 Why Circuits Need Matrices

2 Matrix Operations in Detail

3 Matrix vs. Array Arithmetic

4 Worked Matrix Examples

5 Complex Number Arithmetic

6 Summary and Practice

- Any set of simultaneous linear equations is a single matrix equation  $\mathbf{Ax} = \mathbf{b}$ .
- `np.linalg.solve(A, b)` solves it directly — faster and more reliable than inverting  $\mathbf{A}$ .
- A **singular** matrix (zero determinant) has no unique solution; `solve` raises `LinAlgError`.
- `@` is **matrix** multiplication; `*` is **element-wise**. Confusing them is one of the most common errors in numerical engineering code.
- `A.T` transposes; always check a solution by substitution (`A @ x` should reproduce `b`).

## The real skill

Setting the problem up correctly — not solving it — is where an engineer's judgement matters most.

Before checking with Python, work these out by hand. Assume  $a$  is a scalar,  $b$  and  $c$  are arrays or matrices as appropriate:

```
a + 2 - 3
a + 2 * 3
2 * a**3 + a
a + b
np.dot(b, c)
c.T
np.dot(b, c.T)
b.T + a / 2 + 2
```

```
np.dot(c.T, c)
c**(2**a)
np.dot(c, c)
c * c
b**b
a + b * b**c.T
(a + 2j).T
np.array([a+2j, a+3j]).T
```

This chapter answered how engineers write and solve systems of simultaneous equations compactly.

- Chapter 10 returns to the two-loop circuit on a larger scale: reading many resistor/voltage configurations from a spreadsheet and writing the computed currents back.
- The same  $\mathbf{Ax} = \mathbf{b}$  pattern reappears throughout engineering — structural analysis, circuit networks, control systems.

**Next: Measurements, Files and Spreadsheets**